

SQL	DDL (DB Definition): Creates tables & constraints. DML (Database Manipulation Language): Populate tables. DQL (Data Query Language). DCL (Database Control): Admin & tuning aspects. PL (Programming): Allows to program on the DB server.		
	CREATE TABLE stocks (w_id INTEGER REFERENCES warehouses(w_id) s_qty SMALLINT CHECK(s_qty > 0), PRIMARY KEY (w_id, i_id));		INSERT INTO <table> (col1, col2, ...) VALUES (val1, val2, ...) DELETE FROM <table> WHERE s_id = '394'; UPDATE <table> SET s_city = 'JB' WHERE s_city = ';
	CREATE INDEX i_i_price ON items(i_price); GRANT UPDATE ON stocks TO john;		
	CREATE OR REPLACE FUNCTION myage (dob DATE) RETURNS INTEGER AS \$\$ BEGIN RETURN DATE_PART('year', CURRENT_DATE) - DATE_PART('year', dob)::INTEGER; END; \$\$ LANGUAGE plpgsql; OR for Postgresql: EXTRACT (year FROM AGE(dob)). - Procedures = functions w no return value		
	Constraints Basic: PK, Unique, NOT NULL, FK, CHECK. Advanced: Triggers (more powerful than check as can check from diff tables)		
Queries	Range Query: WHERE ... BTW 0 AND 10; Prefix match query: WHERE ... LIKE 'Si%'; Extremal query: WHERE ... = ALL (SELECT MAX()); Duplicate Elimination: SELECT DISTINCT or UNION. Join query: CROSS JOIN/JOIN/no keyword (no ON, just use WHERE) Natural, Equi, Inner Join query: combine based on equality. INNER JOIN ON NATURAL JOIN (same col) Nested query: uses result of subquery (in SELECT, or FROM, WHERE, HAVING): WHERE <col> = ANY (subquery) Correlated nested query: nested join query & subquery refers to attributes of outer query SELECT s.i_id FROM stocks s WHERE EXISTS (SELECT * FROM warehouses w WHERE s.w_id = w.w_id) Negated queries: (NOT IN, <> ALL, NOT EXISTS, EXCEPT, ...). - Universal Quantification: All items in stock in ALL warehouses SELECT i.i_id FROM items i WHERE NOT EXISTS (SELECT * FROM warehouses w WHERE NOT EXISTS (SELECT * FROM stocks s WHERE s.w_id = w.w_id AND s.i_id = i.i_id));		
CREATE TEMPORARY TABLE <table name> AS ... (exists for duation of DB session)		CREATE VIEW <view> AS ... (unmaterialized views change when base table change. Materialized views need refresh)	WITH <cte name> AS () SELECT ... (exists for query)

ER diagram	Entities = identifiable things. Set of entities of same type = entity set (box). - Usually noun Values = atomic data types [INTEGER, TEXT, ...] (circle). - Derived values of attrs (maybe from other sets) via computation (dashed line) R/s = associate 2 entities. Set of r/s of same type = r/s set (diamond). - Usually verb. - R/s can have attrs. - R/s NOT distinguished by their attrs, but by their participating entities . - Attrs are dependent on the entities being associated		
	R/s can associate entities from the same entity set (2 lines from box to diamond). R/s can associate more than 2 entity sets (n-ary r/s) To solve: "customer can't download same game multiple times on diff device" -> promote attr into entity set (now Customers, Devices, Games w r/s download). But now, w this ternary r/s, seems like Customer download Devices, OR Games download Devices (wrong)		
	In some cases, want to associate an entity set w a r/s set -> wrap r/s set in a box - Want to know what is the primary r/s, which is Customer download Games An aggregate is firstly a r/s set. - Customers cannot download same game multiple times - Downloads must be identified by the participating entities - Other rules relating to identifying attrs also applies		
	1 or more attrs can identify entity. (black circle or connected black circle (for composite)) An entity set can have ≥ 1 set of candidate keys (represent all as separate grouping) Only show minimal key set on ER diagram. Note: minimal ≠ minimum Note: R/s set cannot have black circle (its identified by its entities) Some entities can only be identified within the scope of a r/s w another entity set (connect partial identifier/key to the participation of dominant entity) - Customer cannot be identified by customerid, as have diff app store - Customer is only unique within a single app store Terminology: - Entity set w at least 1 partial key = weak entity set . - Attached to the identifying r/s (e.g. "on") and attached to the dominant entity set (e.g. "App Stores") Variants: - Entity set w partial key can also have other (non-partial) key (still known as weak entity set) - Partial key can have multiple dominant entity set		
	Participation Constraint = num of times entity can appear in r/s (btw entity and r/s set), and restricted by (min, max) value For binary r/s, can classify as follows: (not useful for higher n-ary r/s) (1, _) = mandatory participation. (0, _) = optional participation. (_, 1) = maybe 1-to-1 r/s [if (_, 1) for all entities involved] (_, 1) = maybe 1-to-many r/s [if (_, 1) for 1 but (_, n) or (_, x) for x > 1 for other]. (_, n) = many-to-many r/s If participation constraint omitted, assume default participation is (0, n)		
	Rules 1) Value sets are mapped to domains (meaningful data type). - By default, NOT NULL constraint to be used 2) Entity sets mapped to relations. - Entity set attrs are mapped to attrs of relation - Candidate keys are mapped to PKs and UNIQUE NOT NULL. - <col> <data type> PRIMARY KEY. - PRIMARY KEY (col1, col2) 3) R/s sets are mapped to relations. - Attrs of relation consist of attrs of r/s set. Keys = keys of participating entities (FK)		
	Exception to Rules 1) (0, 1) (Wrong LHS, correct RHS) CREATE TABLE work_for (PK (emp_num, name), FK (emp_num) REFERENCES employees(emp_num), FK (name) REFERENCES companies(name)); Issue: 1 employee can work for 2 diff companies.	CREATE TABLE work_for (PK (emp_num), FK (emp_num) REFERENCES employees(emp_num), FK (name) REFERENCES companies(name)); Restrict PK to entity set w (0, 1) cardinality to avoid issue	
2) (1,1) (Exactly 1) CREATE TABLE work_for (PK (emp_num), FK (emp_num) REFERENCES employees(emp_num), FK (name) REFERENCES companies(name)); Can insert into employee but forget insert to work_for, then (0,1)		CREATE TABLE employee_work_for (PK (emp_num), UNIQUE (email), -- candidate key FK (name) REFERENCES companies(name)); Merge employee and work_for so all employees must work for at least 1 company	
3) Partial Key CREATE TABLE employee_work_for (PK (emp_num), UNIQUE (email), -- candidate key		CREATE TABLE employee_work_for (PK (emp_number, name), UNIQUE (email), -- candidate key FK (name) REFERENCES companies(name)	

	FK (name) REFERENCES companies(name)); This is only partial key; should NOT uniquely identify entity - email is not partial key, and could have been PK	); Add PK of dominant entity set (still need to merge table)
Limitations	Translation scheme tries to minimize num of NULL values, i.e. NULL values not due to translation, but due to meaning of the attr Translation scheme is simple (3 rules + 3 exceptions). But cannot translate other cardinalities (e.g. (1, n)) Enforce as much as possible: - Ensure can uniquely identify entity set. - Ensure max/min cardinality is satisfied.	

OLTP	mainly short transactions involving updates & point queries. -ACID. - Strong consistency	
OLAP	mainly long transactions w complex queries. - Operating on static snapshots of data & approx answers may be acceptable	
OLTP vs OLAP	Doing OLTP and OLAP in same DB system often impractical. - OLAP lock entire table, OLTP cannot write. Soln: data warehouse	
Data Warehouse	Kimball: copy of transaction data for query and analysis. bottom-up, promotes dimensional modelling using star schema - Advocates decentralized data warehouse comprised of data marts (need not be normalized)	
Dimensional Modelling	Data Mart: cube containing individual facts (i.e. line of a receipt) and their measures (i.e. qty sold) of the txs. - Can rearrange the table s.t. each entry in a cell is a single value (or set of values) called measure . Other values become dimensions . ERD: dimensions = entity, fact + measure = r/s. - dim1 = pdt sold, dim2 = date, dim3 = location, measure = qty sold	
Star Schema (1 fact table + several dim tables)	Fact table records its measures aka facts (e.g. qtg, total price, ...). - Fact table records txs at the finest available granularity - Also records surrogate keys of the dimension rows in the diff dimension tables that describe the transaction	
	Dim tables provide description of the dim. Each entry in a dim table has a surrogate key used in the fact table to refer to it (FK) - Surrogate key = synthetic key not derived from app data (unlike natural key)	
	Diff fields of a dim may define hierarchies . E.g. year, quarter, month, week, day hierarchy	
	Info can be repeated in dims for simplicity. Normalization not a concern as data should have been cleaned & no update	
Slice/Dice	Slice = select a layer of the cube by fixing a dim . Dice = select a sub-cube by applying filters on multiple dimensions	
	Conditions & grouping are on attributes of specific dims . [COUNT, SUM, AVG, MIN, MAX]	
	Queries can also use window fns over partitions by specific dims on the natural join of the fact table w dim tables SELECT <dim cols>, <window_fn(measure) OVER (PARTITION BY <dims> ORDER BY <ordering cols>)> FROM ...	
Cube/Roll-Up/Drill-down	Users can use hierarchies defined by diff fields of the dims of the cube to roll-up (less granular) or drill-down (more granular)	
	CUBE = subclause of GROUP BY that defines multiple simultaneous grouping sets. SELECT ... GROUP BY CUBE(col1, col2, ...) CUBE(A, B, C) = (ordered) UNION of query w GROUP BY (A, B, C) + (A, B) + (A, C) + (B, C) + (A) + (B) + (C) + no GROUP BY	
	ROLLUP = subclause of GROUP BY that defines multiple simultaneous grouping sets. SELECT ... GROUP BY ROLLUP(col1, col2, ...) ROLLUP(A, B, C) = (ordered) UNION of query w GROUP BY (A, B, C) + (A, B) + (A) + no GROUP BY	
	Drill-down = no special subclause of GROUP BY. - GROUP BY default is drill down	
Common Facts	Bus architecture = data warehouse containing several fact tables that share common dimensions	
	- CURRENT_DATE. - COALESCE(<cal1>, <default>). - ROUND(<cal1>, <d.p>). - col INT GENERATED ALWAYS AS IDENTITY PK: DB assigns value	

RA	Unary Operation		Selection: σ		Projection: π		Renaming: ρ		
	RA based on relations (based on set). So no duplicate row in RA. But for SQL: multi-set semantics, where duplicate allowed For TEXT: $S1 < S2 = S1$ lexicographically smaller than $S2$. For DATE: $D1 < D2 = D1$ happened before $D2$								
	p		T	F	T	F	2 propositional formulae are equivalent iff have same truth table $\rho(R_1, R_2)$: rename relation R_1 as R_2 . $\rho(R_1, R_2(A_1 \rightarrow A_2))$: rename $R_1.A_1$ to $R_2.A_2$ (or A_2 if no duplicate cols) Union-compatible: 2 relations have same col types for set op to be allowed. For cross product: num of cols = $n + m$; num of rows = $n * m$		
	q		T	T	F	F			
	$p \rightarrow q \equiv \neg(p \wedge \neg q)$		T	T	F	T			
Optimization	All more efficient if intermediate table is smaller								
	Sigma Cascade: $\sigma[\theta_1 \wedge \theta_2](E) \equiv \sigma[\theta_1](\sigma[\theta_2](E)) \equiv \sigma[\theta_2](\sigma[\theta_1](E))$. Can reorder selection & combine into conjunction								
	Selection Distributes over Cross Product: $\sigma[\theta \wedge \theta_1 \wedge \theta_2](E_1 \times E_2) \equiv \sigma[\theta](\sigma[\theta_1](E_1) \times \sigma[\theta_2](E_2))$ - Equivalent only if conditions θ_1 and θ_2 uses only attributes in the respective tables								
	Selection Distributes over Union: $\sigma[\theta](E_1 \cup E_2) \equiv \sigma[\theta](E_1) \cup \sigma[\theta](E_2)$. - Union compatibility = E_1 & E_2 might have diff attribute names.								
	Pi-Cascade: $\pi[L_1](E) \equiv \pi[L_1](\pi[L_2](\dots(\pi[L_n](E))))$. - Equivalent only if $L_1 \subseteq L_2 \subseteq \dots \subseteq L_n$. Else might be error								
	Projection Distributes over Union, Intersection, Difference: $\pi[L](E_1 \cup E_2) \equiv \pi[L](E_1) \cup \pi[L](E_2)$. - Assume union-compatible								
	Typical optimization steps: 1) Cardinality/Cost estimation : estimate size of output or cost of operation (e.g. IO, ...) 2) Path/Query rewriting : explore possible equivalent RA and/or query. 3) Other techniques like index, caching, statistics, ...								

PostgreSQL architecture	Planner/Optimizer: Uses catalog & statistics to devise candidate plans & estimate their respective costs. - Chooses least cost	
	Execution plan: directed acyclic graph (DAG) or a tree of physical algebraic operators . For PostgreSQL, it includes: - Sequeuntial/Index Scans, - Sorting, - Aggregation Operators, - Nested Loops, - Hash/Merge Joins	
	Execution: Total query time = query planning time, execution time, network latency and client overhead	
Explain	EXPLAIN: generates and displays the annotated execution plan. - EXPLAIN (VERBOSE) shows more details - Only supports SELECT, INSERT, UPDATE, DELETE, EXECUTE (of a prepared statement), CREATE TABLE, DECLARE (of a cursor) - EXPLAIN (ANALYZE): executes query and shows execution time and row count for ea step	
	At ea node of the plan, EXPLAIN gives 3 estimates: cost, rows, width 1) cost: estimated startup cost (before any output is produced) & total cost of executing node (& their children) w the query execution plan (CPU & I/O cost). - Expressed in arbitrary units . - Lower costs = more efficient. - Don't include cost of transmission to client 2) rows: estimated num of rows a node return. 3) width: estimated average width (in bytes) of the rows produced by the node - Product of rows and width estimates the total data volume for the result set	
Analyze	- pg_tables: view provides access to useful info (e.g. name, indexes, triggers, ...) about ea table in DB	
	- pg_attribute: catalog stores info about table cols. - Exactly 1 row for every col in every table in DB	
	- pg_statistics: approx statistical data about rows. -- Use ANALYZE; used by query planner. -- NOTE: ANALYZE $\neq$ EXPLAIN ANALYZE	
	- pg_stats: View allows access only to rows of pg_statistic that correspond to tables user has permission to read	
	pg_stats: e.g. most common value, most common frequency, average width (in bytes)	
	EXPLAIN ANALYZE: 1) actual time: avg startup and total execution time of node in milliseconds	
	2) rows: actual num of rows a node returns. 3) loops: num of times node (i.e. the subplan) is executed	
	4) Rows removed by filter: num of rows filtered out by condition. - Pdt of actual time and loops = total time of a node	
	EXPLAIN (ANALYZE) adds overhead to overall time. - Execution time can vary -> average measurements	

	- Costs and actual times are in diff units . - More imp't to look at whether estimated row counts are close EXPLAIN (ANALYZE) also gives planning & execution time (start-up, shut-down time, and time spent processing result rows) Commands VACUUM, ANALYZE, VACUUM ANALYZE reorganizes data and collect statistics. - Costs, times and plans improve
Sequential Scan	If statistics indicate that % of data to retrieve is large or scattered , and if not possible or worth trying to prepare & use another mtd than a sequential scan, then optimizer uses a sequential scan (PostgreSQL default page size = 8KB) Sequential + Sorting (ORDER BY): quicksort, merge sort, heap sort. Sequential + Grouping/DISTINCT (GROUP BY, DISTINCT): diff ops
Index	- Might speed up queries, deletions & updates. - Slows down insertions and updates (since both data and index must be updated) - PG indexes are secondary : stored independently from table. - When searching index, data fetched from both index & table (the heap). - PostgreSQL supports index-only scans . - When query references only cols stored in index (i.e. index is a covering index) PG auto creates a B-tree unique index for ea UNIQUE and PK constraint (not for FK). - The index enforces uniqueness Catalog table and view pg_index and pg_indexes store info about indexes in DB CREATE [UNIQUE] INDEX [name] ON [table_name] [USING method] ({ column_name (expression) }) [WHERE predicate] - UNIQUE checks for duplicate values. - mtd can be btree (default), hash, gist, - predicate defines a partial index
Index Scan	If statistics indicate that % of data to retrieve is tiny & if index available : Optimizer use index scan If stats indicate that % of data to retrieve is average & if index available : Optimizer use bitmap heap scan - Bitmap = in-memory data structure that encodes the pages containing the row. May need AND & OR operations on bitmap Bitmap Index Scan -> Recheck Cond -> Bitmap Heap Scan. - So hybrid btw index and sequential scan Can cluster table using index. - Needs to be done regularly (if there are updates, not auto update). - CLUSTER <table> USING <index> - Physically reorder rows of underlying table to coincide w the key order in the given index. - Table can have only 1 cluster index
Multi-Column	Multicolumn index can be used for index scan. - Multicolumn index can still be used for index scan even w partial condition - Only works if condition involves the prefix of the multicolumn index CREATE INDEX [name] ON [table_name] (col_name) INCLUDE (col1, col2, ...); # Include more cols in index to incr index-only scans - Include cols are not part of search key and used to return values directly in index-only scans

Join	1) Nested Loop Join (NLJ): for every record in outer, find record in inner table and join. - Outer = smaller usually a) Inner Sequential Scan w Temp Table: no indexes, PostgreSQL don't create statistics, only exist for curr conn. - CREATE TEMPORARY TABLE AS - SET enable_material=off; SET enable_hashjoin=off; SET enable_mergejoin=off; b) Materialized Inner Sequential Scan: if inner is sequential scan, subquery, exp op, ..., to not recompute: - RESET enable_material; c) Inner (Bitmap) Index Scan: when inner has multiple filter conditions d) Memoized Inner Index Scan: cache for some records (not entire table like materialized), when outer has repeated join keys 2) Hash Join: if both tables large, no index, equality join, or no statistics available. O(N + M). - RESET enable_hashjoin i) Build phase: for ea row in smaller table, put join_key in hash table. ii) Probe phase: for ea row in larger table, lookup join_key in hash table 3) Merge Join: when join is followed by sorting, or if have index (maybe) Even on ordered table, Hash join may be chosen over Merge join: CREATE TABLE AS SELECT * ORDER BY
VACUUM [FULL]	Recover disk space of updated/deleted rows, update data stats , identify opp for index-only scans, protect against loss of old data - MVCC: maintain multiple versions of data for update and deletes, so tuples only marked dead, not actually deleted
Semi-Join	Check existence. FROM w WHERE EXISTS (SELECT * FROM s WHERE s.id = w.id) or w.w_id IN (SELECT s.w_id); or w.w_id = ANY () a) Nested Loop Semi Join: stop scan when 1st matching row found
Anti (Semi) Join	SELECT ... FROM w WHERE NOT EXISTS (SELECT * FROM s WHERE s.id = w.id) or FROM w LEFT JOIN s ON w.id = s.id WHERE s.id IS NULL or WHERE w.id NOT IN (SELECT s.id FROM s); or WHERE w.id <> ALL () a) Nested Loop Anti Join: include outer row when no matching row found
Join Order	Outer Join: Left/Right Outer Join. - PG may replace Left w Right Join, e.g. Hash Right Join Inner/Cross/Natural Join: optimizer choose 1 amongst several possible join order (can coerce order by reordering tables)
Denormalization	Denormalize by joining some tables tgt. - Writes more complicated and risky since some constraints cannot be maintained - More costly due to manual propagation (e.g. using triggers). - Some but not all queries are faster Views only for convenience. - No performance gain. - Used by optimizer like subquery. Materialized view : middle ground btw a normalized and denormalized schema - Writes more costly, need manual refresh. - REFRESH MATERIALIZED VIEW <view name>. - Materialized views can be indexed
Tips?	PREPARE: query is parsed, analyzed & rewritten. EXECUTE: prepared query is planned and executed. - If prepared statement executed enough times, server may reuse a generic plan rather than re-planning ea time. PREPARE q AS ...; EXECUTE q; DEALLOCATE q; Planner Mtd Config (not recommended): SET enable_seqscan=off; SET enable_bitmapscan=off; SET enable_hashjoin=false; ... Slow queries: - Wrong design (normalized vs denormalized), - Poor config (incr work_mem), - Tuples are scattered, tables, indexes are bloated (VACUUM, CLUSTER, VACUUM FULL, reindexing, ...). - Missing indexes (CREATE INDEX). - PG don't choose best plan (ANALYZE)

Functional Dependency (FDs)	FD σ or the form $X \rightarrow Y$ iff if 2 tuples of r agree on their X-values, then they agree on their Y-values: $r \models X \rightarrow Y \Leftrightarrow \forall t1 \in r: ((t1.X = t2.X) \Rightarrow (t1.Y = t2.Y))$. - Null/empty always true. - Hard to enforce with CHECK -> Use TRIGGER / PK or UNIQUE CREATE OR REPLACE FUNCTION fd() RETURNS TRIGGER AS \$\$ BEGIN IF EXISTS (SELECT ...) THEN RAISE EXCEPTION; END IF; ... CREATE CONSTRAINT TRIGGER tr AFTER INSERT OR ... ON <table> DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION fd(); - But trigger not efficient. - PK or UNIQUE don't work if col not a key (can be achieved via decomposition) $X \rightarrow Y$ is: - trivial iff $Y \subseteq X$. - non-trivial iff $Y \not\subseteq X$. - completely non-trivial iff $Y \neq \emptyset$ AND $Y \cap X = \emptyset$ Class convention allows: - $\emptyset \rightarrow \emptyset$. - $\emptyset \rightarrow Y$. - $X \rightarrow \emptyset$ Given set of FDs Σ , set of attr $S \subseteq R$ is a - superkey iff $S \rightarrow R$. - candidate key / key iff $S \rightarrow R$ AND for all $T \subset S: T \not\rightarrow R$ = minimal superkey. - Prime attr = attr that appears in at least 1 candidate key of R w Σ (else it is a non-prime attr)
Entailment	$X \rightarrow Y$ logically entailed by Σ iff $\forall r$ satisfy all FD $\sigma \in \Sigma$, r also satisfies $X \rightarrow Y$: $\Sigma \models X \rightarrow Y \Leftrightarrow (\forall r: \forall (\forall \sigma \in \Sigma: r \models \sigma) \Rightarrow r \models X \rightarrow Y)$
Armstrong Axioms	1) Reflexivity : $\forall X \subseteq R: \forall Y \subseteq R: ((Y \subseteq X) \Rightarrow (X \rightarrow Y))$. 2) Augmentation : $\forall X \subseteq R: \forall Y \subseteq R: \forall Z \subseteq R: ((X \rightarrow Y) \Rightarrow ((X \cup Y) \rightarrow (Y \cup Z)))$. 3) Transitivity : $\forall X \subseteq R: \forall Y \subseteq R: \forall Z \subseteq R: ((X \rightarrow Y \wedge Y \rightarrow Z) \Rightarrow (X \rightarrow Z))$ If a FD $X \rightarrow Y$ can be derived using Armstrong's axioms from Σ , denote as $\Sigma \vdash X \rightarrow Y$ Thrm: Above 3 inference rule is sound (i.e. correct/valid). Soundness : $\Sigma \vdash X \rightarrow Y \Rightarrow \Sigma \models X \rightarrow Y$ (if can derive it, it is valid/sound) Thrm: The union inference rule is sound: $\forall X \subseteq R: \forall Y \subseteq R: \forall Z \subseteq R: ((X \rightarrow Y \wedge X \rightarrow Z) \Rightarrow (X \rightarrow Y \cup Z))$ The decomposition inference rule is sound: $\forall X \subseteq R: \forall Y \subseteq R: \forall Z \subseteq R: ((X \rightarrow Y \cup Z) \Rightarrow (X \rightarrow Y \wedge X \rightarrow Z))$ Thrm: The Armstrong's axioms are Complete : $\Sigma \models X \rightarrow Y \Rightarrow \Sigma \vdash X \rightarrow Y$ (if it is valid, you can derive it)

	Thrm (Contrapositive): $\sum \not\models X \rightarrow Y \Rightarrow \sum \not\models X \rightarrow Y$ (if you cannot derive it, it is not valid)	
Equivalence	Closure of $\sum$ = set of FDs logically entailed by the FDs in $\sum$: $\sum^+ = \{X \rightarrow Y \mid \sum \models X \rightarrow Y\}$ The Closure of the set of attr $S \subseteq R$ = set of all attributes functionally determined by S: $S^+ = \{A \in R \mid \exists (S \rightarrow \{A\}) \in \sum^+\}$	
	Algo 1: Attr Closure Algo. Input: $S, \sum$. Output: S^+ $\Omega := \sum \# \text{Unused}; \Gamma := S \# \text{Closure}$	while $(X \rightarrow Y)$ and $(X \subseteq \Gamma) \{ \Omega := \Omega - \{X \rightarrow Y\}; \Gamma := \Gamma \cup Y; \}$ return Γ
	$\sum 1$ & $\sum 2$ equivalent iff have the same closure : $\sum 1 \equiv \sum 2 \Leftrightarrow \sum 1^+ = \sum 2^+$ aka $\sum 1$ is a cover of $\sum 2$ (and also $\sum 2$ is a cover of $\sum 1$)	
Minimal Cover	Set of FD $\sum$ is minimal cover iff: 1) RHS of every dependency in $\sum$ is minimal , i.e. of the form $X \rightarrow \{A\}$ 2) LHS of every dependency in $\sum$ is minimal , i.e. for every FD of form $X \rightarrow \{A\}$, there is no FD $Y \rightarrow \{A\}$ in $\sum^+$ s.t. $Y \subset X$ 3) The set itself is minimal , i.e. None of the FD in $\sum$ can be derived from other FDs in $\sum$	
	Algo 2: Minimal Cover Algo. Input: $\sum$. Output: Minimal cover of $\sum$ 1) Simplify RHS of every σ in $\sum$ (decompose)	2) Simplify LHS of every σ in $\sum$: for ea $X \rightarrow \{A\}$, if $\sum \models (X - \{B\} \rightarrow \{A\})$ for some $B \in X$, then replace $X \rightarrow \{A\}$ w $X - \{B\} \rightarrow \{A\}$ 3) Simplify set $\sum$: for ea $X \rightarrow \{A\}$, if $(\sum - (X \rightarrow \{A\})) \models (X \rightarrow \{A\})$, then remove $X \rightarrow \{A\}$ from $\sum$
	A set of FDs $\sum$ is compact iff there is no different FD w the same LHS $\sum$ is canonical cover (compact & minimal): - From minimal cover -> union those w same LHS -> canonical cover	
	Thrm: Every set of FD $\sum$ has at least one minimal cover / compact cover / canonical cover. (All at least 1)	
The Chase	algo that solves the decision problem of whether the FD σ is satisfied by R w $\sum$, i.e. $(R \text{ with } \sum) \models \sigma$. E.g. want to decide $\sum \models (X \rightarrow Y)$	
	1) Setup: Create an instance r of relation schema R w 2 tuples and distinct values for all attributes 2) Initialization: Make all X values the same. 1st row = row of distinguished vars (won't change, can just put all same) 3) Chasing: for ea FD $\in \sum$, if both tuples have same LHS values, make their RHS values the same 4) Check: if target dependency holds = $\sum \models (X \rightarrow Y)$. Note: final table might imply other FD, but is a coincidence (can only ans target)	
	Thrm: The Chase is sound and complete for σ : $R \models \sigma$ is equivalent to $\sum \models \sigma$	
	Thrm: The Chase always terminates for FDs. Thrm: The Chase always produce counter-example if it exists for FDs	

Anomalies	W/o normalization: Unenforced FDs, Redundancy, NULL values, Deletion Anomaly (opp of NULL values)	
Normal Forms	Idea: require every attr fully depends on PK. - Y fully depends on X in $X \rightarrow Y$ if Y is not dependent on any proper subset of X But prime attrs don't fully depend on the composite key. - Idea 1 (refined): Every non-prime attr fully depends on PK. - But can have more than 1 candidate key. - Idea 1 (most refined): Require that every non-prime attr fully depends on ea candidate key (2NF)	
2NF	For every FD $X \rightarrow \{A\} \in \sum^+$: $X \rightarrow \{A\}$ is trivial . OR - A is a prime attr . OR - X is not a proper subset of a candidate key	
NF	Idea 2: No attr is transitively dependent on any candidate key (BCNF) Z is transitively dependent on X in $X \rightarrow Z$ if there is another attr Y that is not a candidate key s.t. $X \rightarrow Y$ and $Y \rightarrow Z$	
BCNF	Relation R w set of FDs $\sum$ is in Boyce-Codd Normal Form (BCNF) if for every FD $X \rightarrow \{A\} \in \sum^+$ - $X \rightarrow \{A\}$ is trivial . OR - X is a superkey . Issue: R = {A, B, C}, $\sum = \{ \{A, B\} \rightarrow \{C\}, \{C\} \rightarrow \{B\} \}$: {C} $\rightarrow$ {B} non-trivial, and {C} not superkey	
NF	Idea 3: Relax BCNF requirements for prime attrs in order to cater for some candidate	
3NF	For every FD $X \rightarrow \{A\} \in \sum^+$: $X \rightarrow \{A\}$ is trivial . OR - A is a prime attr . OR - X is a superkey . BCNF $\subset$ 3NF $\subset$ 2NF	
Normaliza-tion Methods	1) Decomposition mtd. 2) Synthesis mtd. 3) Entity-relationship approach	
	1) Decomposition mtd: assume that DB can be represented by a universal relation (there is a relation that contains all attrs of DB). - Relation is then decomposed into smaller relations called fragments , in order to remove redundant data	
	2) Synthesis Mtd: assume that DB can be described a given set of attrs and a given set of FDs - The 3NF (or occasionally BCNF) relations fragments are then synthesized based on the given set of dependencies	
Decomposi-tion	2 main criterias for decomposition & synthesis mtds: Losslessness (reconstructability) and dependency preserving (covering)	
	A decomposition is a lossless-join decomposition if the natural join (universal relation assumption; assume no NULL values) of all fragments is equivalent to the original relation: $R = R_1 \bowtie R_2 \bowtie \dots \bowtie R_n$. (This is a join dependency) To handle NULL, decomposition is lossless if full outer join of 2 tables on condition that their PK = FK gives back the initial table	
	A decomposition is a dependency preserving decomposition if the info captured by the FDs are captured. i.e. union of the projected sets FDs is equivalent to original set $\sum$: $\sum \equiv (\sum _{R_1} \cup \sum _{R_2} \cup \dots \cup \sum _{R_n})$	
	Lemma 1: Binary Decomposition: Binary decomposition of R is a pair of tables $\delta = \{R_1, R_2\}$ s.t. $R = R_1 \cup R_2$ - It is a lossless-join decomposition if $(R_1 \cap R_2) \rightarrow R_1$ OR $(R_1 \cap R_2) \rightarrow R_2$	
	Lemma 2: General Decomposition: decomposition of R into $R_1, R_2, \dots, R_n$ is a lossless-join decomposition if there exists at least 1 seq of binary lossless-join decomposition that generates the decomposition	
	The Chase for Losslessness: 1) Create a table r w schema R w n tuples. 2) For ea $A \in R_i$, distinguish the A-values in the i^{th} tuple (i.e. set as α). 3) Chase until you find a row of distinguished variables . Then is lossless. If no such row = not lossy	
	Considere relation R w set of FDs $\sum$. A set $\sum'$ of projected FDs on R' from R w $\sum$, where $R' \subseteq R$, is the set of FDs equivalent to the set of FDs $X \rightarrow Y$ in $\sum^+$ s.t. $X \subseteq R'$ and $Y \subseteq R' = \sum _{R'}$, i.e. $X \subseteq R'$: all attrs in X comes from R', $X \rightarrow Y$ in $\sum^+$: FDs can be logically derived from $\sum$	
	Algo 3: Decomposition. Input: $R, \sum$. Output: δ - Works for many NF, - Always terminates - Guaranteed to find lossless-join decomp $R_1 \cap R_2 = X$. $X \rightarrow X^+$ (by defn). $(R_1 \cap R_2) \rightarrow R_1$. $\therefore$ lossless (lemma 1) - May not be dependency preserving	if (R is not in the required NF) then { Let $X \rightarrow Y$ be a violation; Decompose R into 2 relations: $R_1 := X^+$ (wrt $\sum$); $R_2 := (R - X^+) \cup X$ (wrt $\sum$) return { Algo3($R_1, \sum _{R_1}$), Algo3($R_2, \sum _{R_2}$) } else { return { R } }
	Algo 4: Synthesis. Input: $R, \sum$. Output: δ - Works for 3NF (may find BCNF), - Always terminates - Guaranteed to find lossless-join decomp (handled by adding key) - Guaranteed to find dependency preserving decomp Fragments R_i from $(X \cup Y)$. $R_i \models X \rightarrow Y$ (from $\sum^-$) Since this is done $\forall X \rightarrow Y \in \sum^-$, can reconstruct $\sum^-$ So $\sum^- \equiv \sum$ (dependency preserving) - Always use canonical cover (not minimal) to $\downarrow$ num of tables created	$\sum^- = \text{Algo2}(\sum) \#$ Minimal cover; $\delta = \emptyset$ for ea $(X \rightarrow Y \in \sum^-) \{$ if (there is no $R_i \in \delta$ s.t. $(X \cup Y) \subseteq R_i$) then { $\delta = \delta \cup (X \cup Y) \#$ i.e. not subsumed; construct a fragment } } if (there is no $R_i \in \delta$ that contain 1 of the keys) then { add 1 key to the δ } return δ
To enforce FD that was subsumed by Algo 4: Synthesis algo in SQL: R {A, B, D, E} w $\sum \{ \{B, E\} \rightarrow \{A, D\}, \{D, E\} \}$ CREATE TABLE r1 (A INT NOT NULL, B, C, D, PK (B, E), UNIQUE (B, D), FK (D, E) REFERENCES r4(D, E))		CREATE TABLE r4 (D INT NOT NULL UNIQUE , E PK (D, E)